JOUR 5 · COURS · 15 MIN

Le fichier .env et les secrets

Configurer Laravel et protéger les informations sensibles



Projet fil rouge : Mes documents

À la fin de cette séquence

Lire une variable sous la forme NOM=valeur.
Expliquer la configuration de PostgreSQL et Valkey.
Reconnaître les informations sensibles.
Distinguer .env, .env.example et .gitignore.

Des exemples fictifs, sans afficher de secrets

Quel est le rôle de .env ?

C’est un fichier texte de configuration locale.
Il contient des paires nom et valeur.
Ses valeurs aident Laravel à choisir ses paramètres.
Son contenu est généralement lisible en clair.

Le fichier se trouve à la racine du projet Laravel

Une ligne : un nom et une valeur

```
APP_ENV=local
DB_HOST=db
CACHE_STORE=redis
```

Exemple de formation après configuration de Valkey

Du fichier à la configuration

Le fichier .env fournit des valeurs.
Les fichiers du dossier config les utilisent.
Laravel lit ensuite sa configuration pour fonctionner.

Exemple : config/database.php pour la base de données

Configurer PostgreSQL

```
DB_CONNECTION=pgsql
DB_HOST=db
DB_DATABASE=formation_documents
```

Dans Docker, db est le nom du service PostgreSQL

Configurer le cache Valkey

```
CACHE_STORE=redis
REDIS_CLIENT=predis
REDIS_HOST=valkey
```

Ces valeurs sont utilisées une fois le TD Valkey configuré

Quelles valeurs protéger ?

Mot de passe de base de données
Clé de l'application APP_KEY
Token d'API ou clé d'un service externe



Un secret réel ne doit pas être montré ni partagé publiquement

DEUX FICHIERS

.env et .env.example

.env

Configuration propre au poste.
Peut contenir des secrets.
Il reste hors du dépôt Git.

.env.example

Modèle pour préparer le projet.
Valeurs sans secret réel.
Il peut être partagé.

Un exemple de formation ne doit jamais contenir de secret réel

Le rôle de .gitignore

La règle .env évite de proposer un fichier non suivi.
Elle ne chiffre pas le fichier.
Elle ne retire pas un secret déjà suivi ou publié.

Vérifier le contenu à publier avant un commit

Si un secret a été exposé

Prévenir rapidement la personne responsable.
Révoquer ou remplacer le secret exposé.
Vérifier les services puis traiter l’historique Git.

Effacer une ligne dans le dernier commit ne suffit pas

Précautions dans Laravel

APP_DEBUG=false en production.
Éviter de modifier APP_KEY sans raison.
Une configuration mise en cache peut rester ancienne.

Le formateur vérifie la configuration avant toute modification

QUESTION AUX ÉTUDIANTS

Que peut-on partager ?

A Le fichier .env avec un vrai mot de passe
B Un .env.example sans secret réel
C Un token Sanctum valide

Choisissez une réponse et expliquez pourquoi

CORRECTION

Le modèle sans secret peut être partagé

A Non : le mot de passe donne accès à la base.
B Oui : s'il ne contient aucun secret réel.
C Non : le token permet d'agir comme son propriétaire.

La confidentialité dépend du contenu, pas seulement du nom

À RETENIR

Trois réflexes pour le projet

Comprendre ce que configure chaque variable.
Garder les secrets hors de Git et des captures.
Demander de l’aide si une valeur réelle est exposée.

PostgreSQL et Valkey sont configurés selon le poste

Découvrir k6

k6 simule des utilisateurs qui envoient des requêtes.
Il mesure les réponses de notre API.
Un petit script JavaScript décrit le scénario.

L’API Laravel et k6 peuvent fonctionner localement

Ce que nous allons observer

La réaction de l’API avec plusieurs utilisateurs.
Le temps de réponse et le nombre de requêtes.
Les réponses de la protection contre les abus.

Notre scénario concerne la liste des documents

Un VU exécute un scénario

Un VU est un utilisateur virtuel simulé par k6.
Chaque VU répète le scénario pendant le test.
Plusieurs VU peuvent travailler en parallèle.

10 VU ne signifient pas 10 requêtes par seconde

Décrire le comportement

Le scénario indique les actions à effectuer.
La durée indique combien de temps elles se répètent.
Une pause peut représenter le temps de réflexion.

Une itération est une exécution du scénario

Les actions sur notre API

GET /api/documents : lire la liste.
POST /api/documents : créer un document.
La liste protégée demande un token Sanctum.

Le TD de charge lit la liste sans créer de documents

Notre premier scénario

Un utilisateur virtuel pendant 3 secondes.
Chaque lecture envoie le token de formation.
Un check vérifie la réponse HTTP 200.

Docker exécute k6 et Laravel sur le même réseau

Temps de réponse

http_req_duration mesure la durée des requêtes.
avg : durée moyenne.
p(95) : 95 % des mesures sont sous cette valeur.

Comparer des lectures 200 à d’autres lectures 200

Débit et erreurs

http_reqs : nombre de requêtes et débit moyen.
http_req_failed : part des requêtes HTTP en échec.
Un check vérifie une réponse précise.

Dans k6, une réponse 429 compte comme échec HTTP par défaut

Plusieurs utilisateurs virtuels

Essayer successivement 1, 2 puis 5 VU.
Comparer le débit, les durées et les codes HTTP.
Les VU utilisent ici le même compte de formation.

Tous les VU partagent donc la limite de ce compte

Lecture sans cache et avec cache

Sans cache : Laravel interroge PostgreSQL.
Avec cache : une liste déjà chargée peut être réutilisée.
Garder les mêmes données et la même charge.

Quelques mesures locales illustrent le mécanisme

Observer les réponses 429

La liste autorise 5 lectures par minute et par compte.
La requête suivante reçoit HTTP 429.
Après expiration de la fenêtre, une lecture est possible.

Une réponse 429 attendue montre que la limite agit

Interpréter les résultats

Vérifier que le scénario correspond à la question.
Distinguer les lectures réussies des réponses limitées.
Répéter les essais avant de conclure.

Tester uniquement une application autorisée